FaceLit

Documento general del proyecto

Gestión de asistencia académica y reconocimiento facial



Maria José Rodriguez Oyola

Valery Sinaí Trujillo Quintero

Mariana Valenzuela Penagos

SENA

Análisis y Desarrollo de Software

Instructor: José de Jesús Motta

09-03-26

Resumen y contenido

FaceLit es un sistema de gestión de asistencia académica que relaciona usuarios, programas, fichas, ambientes y sesiones de formación con un componente de reconocimiento facial. Su propósito es facilitar el registro y la consulta de asistencia, con responsabilidades diferenciadas para coordinador, instructor y aprendiz.

Este documento presenta el proyecto y describe su implementación. Se elaboró a partir de los repositorios suministrados, el SRS y el documento de despliegue. La revisión del código permite describir estructuras y flujos; no sustituye una prueba de funcionamiento en producción ni acredita el cumplimiento completo de los requisitos.

Palabras clave

Asistencia académica; reconocimiento facial; monolito por módulos; microservicio; PostgreSQL; trazabilidad; Expo; Spring Boot.

Sección	Contenido
1	Introducción, problema y justificación.
2	Objetivos, alcance y población beneficiada.
3	Funcionamiento y tres roles del sistema.
4	Documentación: SRS, HU, MoSCoW y modelos.
5	Base de datos: organización, tablas y relaciones.
6	Backend: arquitectura, seguridad y configuración.
7	Frontend: pantallas, integración y flujo completo.
8	Reconocimiento facial y despliegue.
9	Resultados esperados y conclusiones.

Terminología del sistema

Usar “ficha” para el grupo de formación, aunque el código lo denomina chip; “sesión” para el contexto de registro de asistencia; “enrolamiento” para registrar el rostro y “verificación” para compararlo. La carpeta admin del frontend corresponde al coordinador: no representa un cuarto rol.

1. Introducción y problema

Introducción

La asistencia es una información necesaria para el seguimiento de la formación. Cuando su registro depende de listados manuales o de herramientas desconectadas, resulta difícil mantener datos oportunos, identificar inconsistencias y consultar el historial de cada aprendiz. FaceLit propone reunir ese proceso en una aplicación con información académica centralizada y registro asistido por reconocimiento facial.

Planteamiento del problema

El problema que orienta el proyecto es la necesidad de registrar y consultar asistencia de manera organizada, vinculando cada evento con el aprendiz y la sesión correspondiente. Los procedimientos manuales pueden generar omisiones, duplicados, demoras y dificultad para verificar quién realizó el registro. Estos riesgos motivan la solución; no se presentan aquí como estadísticas medidas en una institución.

Además del reconocimiento, el proceso necesita reglas: quién puede administrar usuarios, qué fichas corresponden a cada instructor, cuándo se admite el registro y cómo consulta el aprendiz sus propios datos. Una cámara por sí sola no resuelve esas necesidades.

Justificación

FaceLit integra la gestión académica y la identificación facial para reducir pasos manuales, mejorar la organización de los registros y facilitar su consulta por rol. La separación del componente facial en Python permite manejar sus modelos y dependencias de forma independiente del backend Java. PostgreSQL conserva relaciones y restricciones para mantener la coherencia entre usuarios, fichas y eventos.

El valor académico del proyecto también está en aplicar análisis de requisitos, diseño de datos, desarrollo de APIs, interfaces, seguridad y despliegue. Las mejoras deben comprobarse con escenarios de uso y evidencia, sin atribuir al reconocimiento facial una precisión absoluta ni una protección contra toda suplantación.

2. Objetivos, alcance y beneficiarios

Objetivo general

Desarrollar un sistema para gestionar la asistencia académica mediante funciones diferenciadas por rol e integración de reconocimiento facial, relacionando los registros con los aprendices y las sesiones de formación.

Objetivos específicos

1. Organizar usuarios, programas, fichas y asociaciones académicas en una base de datos relacional. 2. Establecer acceso autenticado y permisos para coordinador, instructor y aprendiz. 3. Integrar el enrolamiento y la verificación facial mediante un microservicio Python. 4. Facilitar la configuración de sesiones y la consulta de registros de asistencia. 5. Ofrecer una interfaz compartida para web y Android. 6. Documentar la solución para su operación, mantenimiento y evaluación.

Alcance

La implementación revisada incluye gestión académica y de usuarios, acceso y perfil, funciones de ambientes y sesiones, registro facial, captura y consulta de asistencia, notificaciones y componentes de actualización en tiempo real. La existencia de una pantalla o endpoint no demuestra que todo su recorrido esté validado: algunos requisitos del SRS requieren comprobación adicional.

El documento no presenta la gestión de horarios como módulo activo. Tampoco incorpora Raspberry Pi: el procesamiento facial se plantea mediante un microservicio Python. No se describe FaceLit como un sistema completo de administración educativa, calificaciones o gestión financiera.

Población beneficiada	Beneficio previsto
Aprendices	Consultar información propia y participar en el proceso de registro facial y asistencia.
Instructores	Gestionar el contexto de sus sesiones y revisar los registros de sus aprendices.
Coordinación académica	Administrar información y asociaciones para organizar el seguimiento.
Equipo de desarrollo y soporte	Contar con una solución documentada que pueda mantenerse y mejorarse.

3. Qué hace cada rol

Rol	Responsabilidad	Interacción facial
Coordinador	Administra usuarios y datos académicos; organiza asociaciones de programas, fichas e instructores. Dispone de funciones de corrección y gestión de solicitudes.	Interviene en la gestión de solicitudes de restablecimiento del registro facial, según permisos y flujo disponible.
Instructor	Trabaja con las fichas asignadas y configura el contexto de ambientes y sesiones; consulta y opera la asistencia.	Utiliza la captura en una sesión y revisa el resultado de identificación y registro.
Aprendiz	Consulta su perfil e información de asistencia y participa en el registro de su rostro.	Realiza el enrolamiento inicial y dispone de solicitud de restablecimiento cuando necesita actualizarlo.

Separación de responsabilidades

El coordinador organiza y administra; el instructor opera el proceso de la sesión; el aprendiz aporta su registro facial y consulta sus datos. El reconocimiento es una función técnica del sistema, no un rol humano adicional. Un aprendiz no debería poder consultar o modificar datos de otra persona por cambiar una URL.

Precisión sobre solicitudes y notificaciones

El frontend contiene funciones y estados relacionados con solicitudes y notificaciones. Algunas acciones utilizan estado local. Para afirmar que una solicitud pasa de una cuenta a otra y persiste, se debe comprobar el recorrido completo contra la API y la base de datos; la interfaz aislada no basta como evidencia.

Ejemplo narrado

Primero, coordinación organiza los usuarios y las asociaciones académicas. Después, el aprendiz registra su rostro. El instructor prepara una sesión vinculada al contexto académico y utiliza la captura facial. El sistema compara la imagen con candidatos activos de la ficha y devuelve un resultado; el proceso de asistencia registra el evento conforme a las reglas de la sesión. Finalmente, cada rol consulta la información que le corresponde.



4. Documentación del proyecto

Documento	Qué responde	Cómo se usa
SRS	¿Qué debe hacer el sistema y bajo qué condiciones?	Define actores, requisitos funcionales y no funcionales, reglas y criterios. Se contrasta con la implementación.
Historias de usuario	¿Qué necesita un rol y para qué?	Expresan necesidades pequeñas y criterios de aceptación verificables.
MoSCoW	¿Qué se prioriza en una entrega?	Clasifica Must, Should, Could y Won’t para ese alcance.
Diagramas	¿Cómo se relacionan actores, procesos y componentes?	Explican casos de uso, actividades, secuencias y estructura.
MER	¿Qué entidades y relaciones representan los datos?	Orienta claves, cardinalidades y organización persistente.
Modelo de dominio	¿Qué conceptos y reglas tiene el negocio?	Explica aprendiz, ficha, sesión y asistencia sin depender de una pantalla.

Cómo conectar requisitos y código

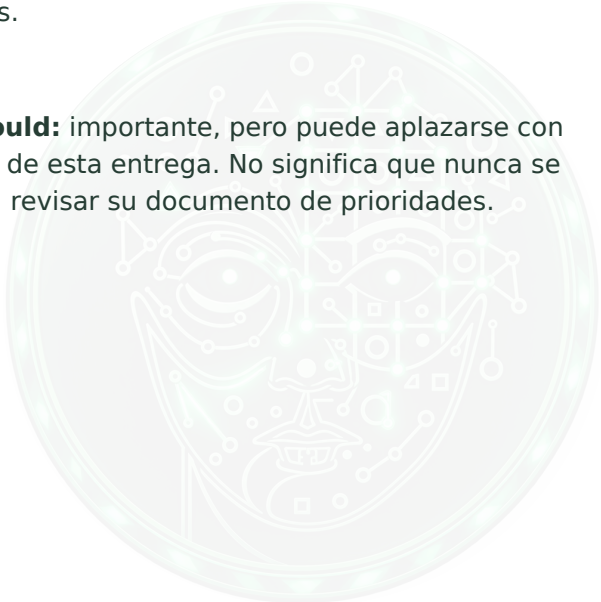
Una trazabilidad útil vincula necesidad → historia o requisito → pantalla → endpoint → servicio → tabla → evidencia de aceptación. Ejemplo de estudio: “consultar mi asistencia” debe relacionarse con el rol aprendiz, la vista correspondiente, el servicio de consulta y sus restricciones de acceso. Este ejemplo es didáctico; no reemplaza los identificadores oficiales de las historias del equipo.

Ejemplo de historia de usuario

Como aprendiz, quiero consultar mis registros de asistencia para conocer mi participación en las sesiones. Criterios sugeridos: con sesión válida se muestran únicamente mis registros; sin registros se informa un estado vacío; ante un fallo de conexión se muestra un mensaje y se permite reintentar. Deben ajustarse a las HU existentes.

Priorización MoSCoW

Must: indispensable para entregar el flujo central. **Should:** importante, pero puede aplazarse con una alternativa. **Could:** mejora opcional. **Won’t:** fuera de esta entrega. No significa que nunca se hará. No se atribuye una matriz aprobada al equipo sin revisar su documento de prioridades.



4.1. Diagramas y organización del sistema

Modelo	Descripción
Casos de uso	Qué funciones puede iniciar cada uno de los tres roles.
Actividad	Pasos, decisiones y alternativas: permisos de cámara, captura y respuesta.
Secuencia	Orden de mensajes entre frontend, backend, microservicio y base de datos.
MER / relacional	Entidades, claves y cardinalidades; no describe el orden de una solicitud.
Dominio	Conceptos y reglas del negocio; no es una copia del diseño de pantallas.

Monolito y módulos: la explicación correcta

El backend Java es un monolito porque se compila y ejecuta como una sola aplicación Spring Boot. Está organizado por módulos funcionales y capas internas. Ambos términos son compatibles: uno describe la unidad de ejecución y el otro la organización del código. No se observaron submódulos Maven ni una garantía formal de aislamiento entre paquetes.

El sistema completo contiene, además, un microservicio facial independiente. Por eso no conviene decir que todo FaceLit es un único monolito ni que cada paquete Java es un microservicio. Tampoco basta la presencia de carpetas para afirmar DDD completo o arquitectura hexagonal.

5. Base de datos: herramientas y organización

Herramienta	Para qué se utiliza
PostgreSQL 16	Motor relacional para persistir usuarios, datos académicos y eventos.
pgvector	Tipo vector y operaciones para comparar embeddings faciales.
Liquibase 5.0.2	Aplica y registra cambios versionados del esquema y datos iniciales.
Docker Compose	Levanta PostgreSQL local y el servicio de migración.
SQL / UUID	Define objetos y consultas; UUID identifica registros sin un contador simple.

¿Para qué funciona Liquibase?

Liquibase no es el motor de base de datos ni una librería de reconocimiento facial. Lee el changelog y ejecuta los changesets pendientes en PostgreSQL. Registra su ejecución en DATABASECHANGELOG y utiliza control de bloqueo; los checksums ayudan a detectar cambios en migraciones ya aplicadas. Permite repetir el proceso de preparación del esquema de forma controlada.

En este proyecto, Liquibase se ejecuta por separado del backend. Spring configura liquibase.enabled: false y Hibernate ddl-auto: validate: Java comprueba que el esquema coincide con las entidades, pero no lo crea automáticamente. La secuencia es preparar PostgreSQL, aplicar migraciones y arrancar el backend.

Organización	Contenido
01_ddl/	Extensiones, esquemas, tablas, alteraciones, claves e índices.
02_dml/	Datos iniciales y cambios de datos.
03_dcl/ y 04_tcl/	Organización de scripts de permisos y transacciones; revisar los archivos efectivos.
05_rollbacks/	Reversiones disponibles, cuya cobertura debe comprobarse.
changelog-master.yaml	Entrada principal de migraciones.
docker/ y scripts/	Apoyo a construcción y ejecución.

5.1. Esquemas, tablas y relaciones

La base utiliza los esquemas security, roleandpermission, academic, environment, notification, legal y facialrecognition. Un esquema es un espacio de organización dentro de la base: no equivale a un servidor separado ni a un microservicio.

Ejemplo: academic.chip representa una ficha

Campo	Tipo / regla	Significado
id_chip	UUID, clave primaria	Identificador de la ficha.
id_program	UUID, obligatorio, clave foránea	Programa al que pertenece.
chip_code	VARCHAR(20), obligatorio, único	Código visible de la ficha.
start_date / end_date	DATE	Fechas de la ficha.
state	VARCHAR(20), obligatorio	Estado del registro.
created_at / updated_at	Marca de tiempo	Auditoría de creación y actualización.
deleted_at / deleted_by	Campos de auditoría	Permiten representar eliminación lógica.

Relaciones principales

Relación	Cómo se representa
Programa → fichas	Una ficha tiene id_program; un programa puede tener varias fichas.
Instructor ↔ fichas	instructor_chip es la tabla de asociación.
Aprendiz ↔ fichas	apprentice_chip relaciona aprendiz y ficha e incorpora estado.
Usuario → instructor / aprendiz	Las entidades académicas enlazan con security.user_app.
Aprendiz → rostro	user_face contiene id_apprentice único y el embedding facial.
Sesión → eventos	facial_event referencia el contexto record_environment.

La asociación de aprendices conserva contexto e historial. No debe deducirse que se permiten varias fichas activas simultáneas únicamente por existir una tabla intermedia: las reglas de negocio también intervienen.

5.2. Datos faciales e integridad

¿Qué es un embedding?

Es un vector numérico que el modelo obtiene de un rostro. No es una contraseña ni el nombre de la persona. La tabla facialrecognition.user_face conserva el vector y metadatos como dimensión, modelo, referencia de fotografía y estado. El código de migración revisado mantiene el tipo vector de pgvector.

De una captura a una comparación

Python recibe la imagen, la decodifica y genera un embedding. En la verificación de una sesión, busca candidatos activos vinculados a su ficha. PostgreSQL calcula distancia coseno entre vectores y el servicio la convierte en similitud para comparar contra un umbral. El resultado de similitud no equivale a una probabilidad de identidad.

Claves y restricciones

La clave primaria identifica una fila; la clave foránea enlaza una fila con otra tabla; UNIQUE impide repetir valores que deben ser únicos; NOT NULL exige un dato; los índices ayudan a localizar y restringir registros. Las reglas del backend complementan estas restricciones, pero no las reemplazan.

El script 008_unique_facial_event_attendance.sql trata eventos duplicados y crea un índice único parcial para los eventos no eliminados por sesión, aprendiz y tipo. Es una medida de integridad observable, sin atribuirle tiempos de mejora no medidos.

Buenas prácticas de mantenimiento

Agregar nuevas migraciones en lugar de modificar changesets ya ejecutados; verificar el entorno destino; respaldar antes de cambios sensibles; revisar la restauración; limitar permisos de acceso a información biométrica. Si una tabla contiene deleted_at, las consultas deben considerar el estado de eliminación lógica cuando corresponda.

6. Backend arquitectura y capas

El backend Java se organiza como un monolito modular con arquitectura en capas o N capas. Los módulos se agrupan por funcionalidad, pero se compilan y ejecutan juntos en una aplicación Spring Boot. N capas describe la separación lógica de responsabilidades; no significa que cada capa tenga un servidor independiente.

Organización por funcionalidad

auth/ reúne seguridad, login, roles y usuarios; academic/ reúne programas, fichas, instructores y carga CSV. config/ concentra seguridad transversal; shared/ aporta constantes, auditoría, utilidades y manejo común de errores. La documentación aportada se centra en estos módulos. El sistema también contiene funciones de ambientes, reconocimiento facial, notificaciones y tiempo real descritas en las demás secciones.

Capas dentro de cada módulo

controller/ recibe solicitudes HTTP y delega. service/ define operaciones del negocio y serviceImpl/ las implementa. repository/ consulta y persiste mediante JPA. model/ contiene entidades, enums y converters. dto/request y dto/response definen el contrato de entrada y salida. exception/ identifica errores del módulo. Modelos y DTOs son estructuras utilizadas por las capas; no son pasos obligatorios separados de una solicitud.

Una operación habitual sigue controlador → servicio → repositorio → PostgreSQL. Por ejemplo, ProgramController delega en ProgramServiceImpl, que valida y coordina ProgramRepository y la bitácora. El controlador devuelve la respuesta sin asumir la persistencia.

Por qué se utiliza esta arquitectura

La organización facilita localizar cada función, separar interfaz HTTP, reglas y datos, y mantener una unidad de despliegue Java. Resulta adecuada para el alcance del proyecto sin introducir servicios separados para cada función académica. La existencia de paquetes no garantiza aislamiento estricto entre módulos ni demuestra una arquitectura hexagonal.

El microservicio Python sigue siendo independiente del monolito Java. Las referencias a schedule no se consideran un módulo activo. No se concluye que todos los paquetes de ambientes estén vacíos a partir de un ejemplo parcial de estructura.

6.1. Patrones del backend

Los patrones siguientes se explican según la documentación técnica V5 suministrada. Los nombres de clase permiten localizar su aplicación; esta actualización no constituye una nueva ejecución o auditoría de ese código.

Singleton gestionado por Spring

Reutiliza una instancia de cada bean por defecto dentro del contenedor. Se aplica con @Service, @Repository, @Component y @Configuration: ProgramServiceImpl, JwtServiceImpl, EmailServiceImpl, JwtFilter y SecurityConfig. Evita crear componentes manualmente; los servicios no deben guardar estado mutable de una petición en sus campos.

Factory Method métodos de fábrica

LoginResponseDTO.success(...), AssignRoleResponseDTO.success(...) y UserConfigurationResponseDTO.created(...) / updated(...) construyen respuestas uniformes y evitan repetir su inicialización. Son fábricas estáticas de DTOs; no una jerarquía de creadores del Factory Method GoF estricto. Se encuentran en auth/dto/response/.

Builder

Jwts.builder() construye el token paso a paso en auth/service/serviceImpl/roleandpermission/JwtServiceImpl.java. Es un builder de librería. Lombok genera código repetitivo, pero getters y constructores no implican Builder: solo @Builder o una API equivalente justifican ese nombre.

Facade

ProgramServiceImpl, ChipServiceImpl, InstructorServiceImpl, UserChipServiceImpl y CsvAcademicServiceImpl ofrecen un punto de entrada al caso de uso. En academic/service/serviceImpl/academic/ coordinan validaciones, repositorios y bitácora; los controllers delegan. CsvAcademicServiceImpl orquesta la carga CSV.

Adapter

Los DTOs adaptan datos internos al contrato público sin exponer entidades JPA completas. InstructorTypeConverter, en academic/model/converter/, realiza una adaptación específica: ESPECIFICO se guarda como SPECIFIC y TRANSVERSAL como CROSS-CUTTING, compatibles con la restricción de la base.

Proxy o interceptor de seguridad

config/JwtFilter.java intercepta solicitudes, valida el JWT y prepara SecurityContextHolder antes del controlador. SecurityConfig lo incorpora a la cadena de filtros. Es un interceptor de seguridad, no un Proxy GoF puro.

Herencia de auditoría

shared/model/AuditBase.java usa @MappedSuperclass para compartir campos con Program, Chip, Instructor, User y Credential. Hibernate actualiza createdAt y updatedAt; createdBy y updatedBy no se vinculan automáticamente al JWT. Es reutilización por herencia JPA, no Template Method estricto.

6.2. JWT autorización y correo

Autenticación con JWT

El acceso habitual utiliza documento y contraseña. El servicio revisado también admite búsqueda por correo como compatibilidad. Comprueba el estado de la cuenta y la contraseña con BCrypt. Cuando el acceso es válido, genera un JWT firmado y devuelve la información necesaria para la sesión.

El frontend añade Authorization: Bearer <token> al llamar al backend. Un filtro valida el token y establece la identidad para la solicitud. La configuración es stateless: el servidor no depende de una sesión HTTP tradicional para cada petición. El JWT está firmado; no debe describirse como una contraseña cifrada ni como un mecanismo que cifra toda la comunicación.

Autorización

Autenticar responde “¿quién eres?”; autorizar responde “¿qué puedes hacer?”. SecurityConfig limita rutas por rol y exige autenticación en otras. También se deben revisar controles sobre el recurso concreto: estar autenticado no da derecho a modificar cualquier aprendiz o ficha. El canal de tiempo real dispone de autenticación mediante token en su intercambio inicial.

Notificaciones por correo electrónico

shared/service/impl/EmailServiceImpl.java utiliza JavaMailSender y SimpleMailMessage. PasswordRecoveryServiceImpl genera un código con vigencia de cinco minutos y solicita su envío. El correo de recuperación es distinto de las notificaciones del dominio.

Errores y comprobación

NotificationServiceImpl guarda la notificación y, para APP_EMAIL, publica EmailDispatchRequested. EmailDispatchServiceImpl escucha después del commit y envía con @Async; registra intentos y estados. Según la documentación aportada, un trabajo revisa pendientes y fallidos cada cinco minutos hasta tres intentos. BackendApplication habilita ejecución asíncrona y tareas programadas. La recepción real depende también de SMTP y del despliegue.

6.3. Configuración del entorno

El archivo del repositorio se llama application.yaml. YAML admite también la extensión .yml: ambas expresan el mismo formato. Centraliza configuración de base de datos, servidor, correo, JWT e integración facial. No es una pantalla del sistema ni un lugar para publicar contraseñas.

Variable / propiedad	Uso
DB_URL, DB_USERNAME, DB_PASSWORD	URL JDBC y credenciales PostgreSQL del backend.
JWT_SECRET	Clave para firmar/verificar tokens; mantener en el servidor.
MAIL_USERNAME, MAIL_PASSWORD	Credenciales del servicio de correo.
FACIAL_EMBEDDING_URL	Dirección del servicio Python vista desde Java.
FACIAL_EMBEDDING_API_KEY	Credencial usada por el cliente de servicio facial.
EXPO_PUBLIC_API_URL	URL del backend incorporada al frontend.
EXPO_PUBLIC_EMBEDDING_API_URL	URL del microservicio usada por el cliente facial directo.

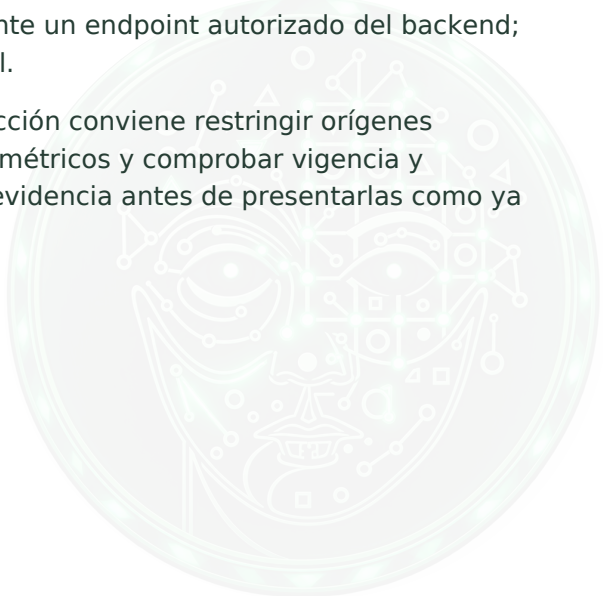
Valores observados

La configuración Java declara puerto 8080, expiración JWT de 28.800.000 ms (8 horas), Liquibase desactivado y ddl-auto: validate. Se configura SMTP en el puerto 587 y cargas multipart de hasta 20 MB. El archivo no enlaza automáticamente el puerto a una variable PORT: comprobar el artefacto desplegado antes de afirmar lo contrario. Spring sustituye referencias como ${DB_URL} al iniciar. spring-dotenv permite cargar .env local; en despliegue se configuran variables del proceso o plataforma.

Público no significa secreto

Las variables EXPO_PUBLIC se incorporan al cliente y pueden ser accesibles a quien use la aplicación. La clave facial incluida en el frontend no puede tratarse como secreto de servidor. Una mejora de seguridad es mediar el enrolamiento mediante un endpoint autorizado del backend; esto se identifica como propuesta, no como flujo actual.

La configuración CORS revisada es amplia. Para producción conviene restringir orígenes autorizados, usar HTTPS, controlar accesos a datos biométricos y comprobar vigencia y almacenamiento de tokens. Estas acciones requieren evidencia antes de presentarlas como ya realizadas.



7. Frontend: pantallas y arquitectura

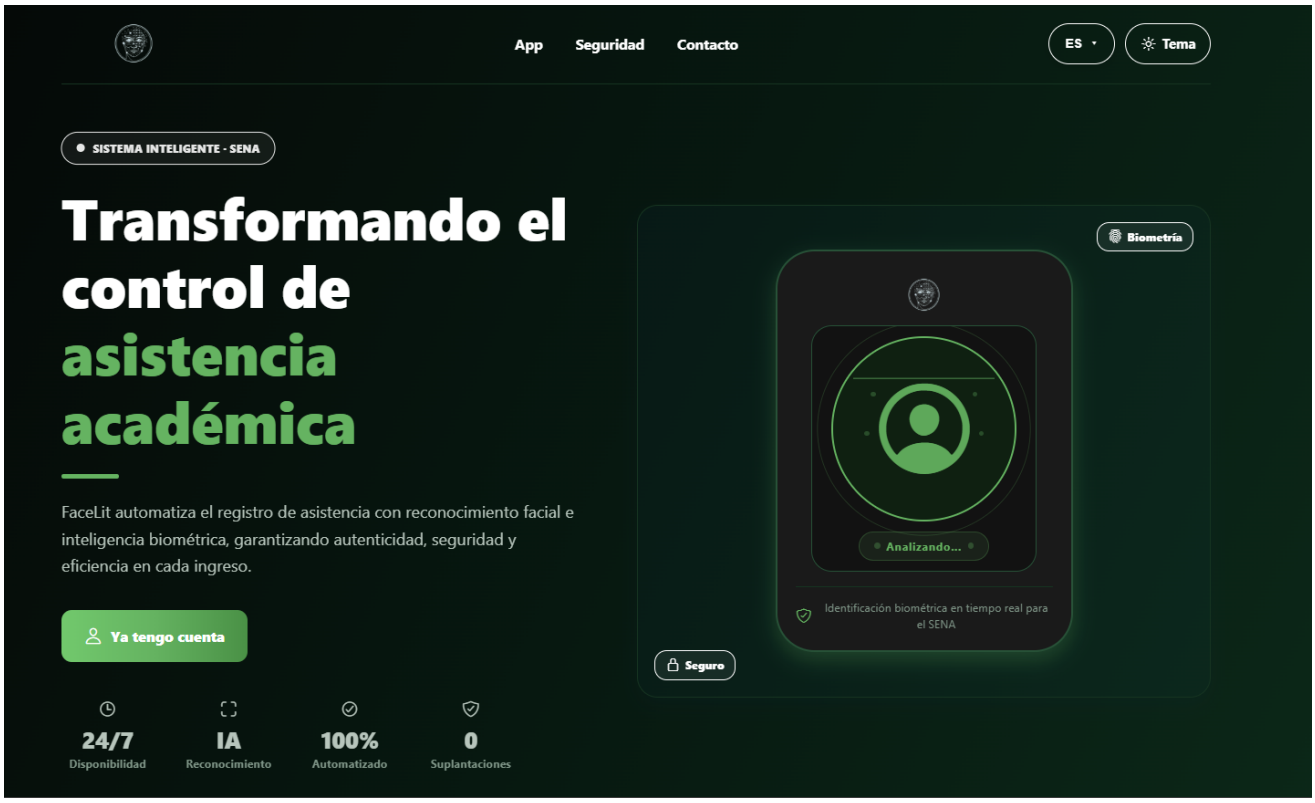


Figura 1. Pantalla de presentación proporcionada por el equipo. Los mensajes promocionales de la imagen no se utilizan como métricas verificadas del proyecto.

Organización de la aplicación

El frontend combina organización por funcionalidades y separación de responsabilidades. Expo Router organiza pantallas en app/; features/ contiene hooks, tipos, estado y componentes de cada función; shared/ reúne componentes reutilizables, contextos, constantes y servicios. React Native, Expo y TypeScript permiten compartir código entre web y Android. Esta estructura facilita reutilizar interfaz y mantener la lógica fuera de las pantallas; no es una réplica de las capas JPA del backend.

Área de pantallas	Función
Acceso y recuperación	Entrar al sistema y utilizar el flujo de recuperación disponible.
Coordinador: app/admin	Gestión de usuarios, información y asociaciones académicas.
Instructor	Contexto de fichas, ambientes, sesiones y asistencia.
Aprendiz	Perfil, registro facial y consulta de asistencia propia.
Elementos compartidos	Perfil, notificaciones, estados de carga y mensajes.

La navegación debe acompañar los permisos del servidor. Ocultar un botón mejora la experiencia, pero no sustituye la autorización backend.

7.1. Frontend responsabilidades y flujo

Pantallas hooks contextos y servicios

Las pantallas reciben acciones y muestran resultados. Los hooks controlan formularios, validaciones, carga y lógica de una función. Los contextos comparten estado entre pantallas; los stores o hooks del módulo mantienen datos específicos. Los servicios realizan solicitudes HTTP. No todas las operaciones pasan por un contexto: algunos hooks llaman directamente a un servicio.

Dónde se aplica en FaceLit

app/auth/login.tsx presenta el acceso; features/auth/hooks/useLoginForm.ts controla el formulario; shared/contexts/AuthContext.tsx gestiona usuario, rol e inicio/cierre de sesión; shared/services/api.js configura Axios. ThemeContext comparte el tema e I18nContext el idioma. shared/constants/routes.ts centraliza rutas y Expo Router resuelve la navegación.

Inicio de sesión

El usuario escribe documento y contraseña. La pantalla utiliza useLoginForm, que valida campos y privacidad, y llama a AuthContext.login(...). El servicio envía POST /api/auth/login. Java valida credenciales y devuelve token y rol; el cliente almacena el token, actualiza el usuario y abre el panel del coordinador, instructor o aprendiz.

Comunicación y configuración

Axios añade Authorization: Bearer al llamar a Java. EXPO_PUBLIC_API_URL define el destino: localhost:8080 puede servir en web local; 10.0.2.2:8080 en el emulador Android; un teléfono físico requiere una dirección de red alcanzable. En producción se utiliza la URL desplegada. Las variables EXPO_PUBLIC forman parte del cliente y no deben guardar secretos privados.

Flujo completo y reconocimiento facial

El usuario actúa en la interfaz; un hook o contexto coordina el servicio HTTP; Java aplica permisos y reglas y consulta PostgreSQL; la respuesta actualiza el estado y la pantalla. El enrolamiento facial revisado utiliza un cliente directo a Python configurado con EXPO_PUBLIC_EMBEDDING_API_URL. La verificación de asistencia por sesión dispone de un cliente Java hacia Python. Registrar un embedding no equivale a registrar asistencia.

Respuesta estado y traducciones

Solo una respuesta confirmada debe mostrarse como éxito. Ante rechazo o desconexión, se conserva un estado coherente y se comunica el error. Algunas pantallas usan datos locales o de prueba y no demuestran integración completa. i18next y react-i18next organizan traducciones es, en, de y fr. Los permisos de cámara y la captura se gestionan según web o móvil.

8. Microservicio de reconocimiento facial

El servicio separado utiliza Python, FastAPI, InsightFace, ONNX Runtime, OpenCV, NumPy, SQLAlchemy y asyncpg. Su estructura distingue rutas API, esquemas de entrada/salida, servicios, repositorios, trabajador facial y configuración de base de datos. No necesita Raspberry Pi para la arquitectura descrita.

Qué hace el modelo

El trabajador usa FaceAnalysis con el modelo buffalo_l y ejecución en CPU. Decodifica la imagen y exige exactamente un rostro para generar el embedding. El registro vincula el vector al aprendiz; la verificación obtiene un vector temporal y realiza una búsqueda limitada por el contexto de sesión y los candidatos activos.

Operación	Contrato observado
POST /api/v1/facial-embeddings/from-image	Recibe user_id e image_base64 para el registro facial.
Verificación de sesión: verify-session	Recibe record_environment_id, imagen y un umbral opcional.
Resultado de comparación	Indica coincidencia, aprendiz cuando aplica, similitud, umbral, modelo y motivo.

Cómo interpretar una respuesta

Un resultado sin coincidencia puede deberse a similitud insuficiente o ausencia de candidatos activos. El umbral por defecto observado es 0,65; requiere validación con datos representativos para valorar falsos positivos y negativos. No se presenta como una precisión del 65 %.

Límites que conviene explicar con honestidad

Las comprobaciones de calidad visual o movimiento no demuestran por sí solas una protección completa contra fotografías o vídeos. El SRS puede pedir medidas que aún deben contrastarse con la implementación. La comparación facial necesita pruebas con iluminación, cámaras y usuarios reales, además de revisar permisos y protección de datos.

8.1. Despliegue: herramientas y proceso

FaceLit está desplegado íntegramente en un VPS de Hostinger y se accede desde https://facelit.tech/. Docker Compose coordina web, backend, servicio facial y PostgreSQL. La publicación del 4 de noviembre se preparó desde main local de los cuatro repositorios, con los ajustes registrados en el manifiesto. Actualización documental: 5 de noviembre de 2026.

Componente	Herramienta / destino vigente
Base de datos	PostgreSQL 16 con pgvector en la red privada Docker del VPS; migraciones Liquibase.
Backend	Spring Boot en un contenedor privado del VPS Hostinger; Caddy conecta la API y WebSocket.
Web	Exportación web de Expo servida por Caddy en https://facelit.tech/, dentro del mismo VPS.
Android	EAS genera el APK por separado; su perfil debe configurar la API de Hostinger antes de reconstruirlo.
Microservicio Python	FastAPI desplegado en un contenedor privado del mismo VPS, conectado al backend y a PostgreSQL.

Orden del proceso

1. Preparar el paquete desde los cuatro checkouts locales de main, registrar SHA y ajustes en el manifiesto y respaldar la base.

2. Transferir el paquete al VPS y configurar dominio y variables privadas del entorno.

3. Iniciar PostgreSQL, validar el changelog y aplicar migraciones con Liquibase.

4. Construir y activar los contenedores de FastAPI, Spring Boot y web con Docker Compose.

5. Publicar la exportación web con Caddy, HTTPS y proxy de API/WebSocket en facelit.tech.

6. Comprobar acceso, CSV, correo, cámara y asistencia; conservar manifiesto, respaldo y resultados del release.

Direcciones y puertos

Solo Caddy publica los puertos 80 y 443 del VPS. PostgreSQL, Spring Boot y FastAPI se comunican por la red interna Docker y no exponen sus puertos al exterior. El puerto 5439 del Compose local es una configuración de desarrollo.

El equipo confirma que login, carga CSV, correo SMTP, cámara y asistencia funcionan. El informe de despliegue detalla el procedimiento, las versiones y las diferencias entre el paquete instalado y GitHub.

9. Tecnologías y resultados esperados

Tecnología	Justificación en FaceLit
Java 17 / Spring Boot 3.4.1	Servicios, seguridad y acceso a datos dentro de una aplicación Java.
Spring Data JPA / Hibernate	Mapeo entre entidades y tablas, repositorios y transacciones.
PostgreSQL 16 / pgvector	Relaciones académicas y comparación de vectores faciales.
Liquibase 5.0.2	Migraciones reproducibles y registro de cambios.
Expo / React Native / TypeScript	Interfaz compartida, rutas y componentes para web y Android.
Axios / i18next	Intercambio HTTP y recursos de traducción.
FastAPI / InsightFace / ONNX Runtime	API Python y generación de embeddings del rostro.
Docker Compose / Caddy / Hostinger / EAS	VPS para la publicación web y servicios; EAS para generar APK por separado.

Las versiones se toman de declaraciones y configuración del repositorio, no de una instalación validada. En Python se declara FastAPI 0.115.6 e InsightFace 0.7.3; el Dockerfile usa Python 3.12. Conservar los archivos de dependencias y comprobar compatibilidad al reconstruir el entorno.

Resultados esperados

Se espera centralizar información de asistencia, reducir omisiones y duplicados, facilitar consultas por rol y mantener trazabilidad entre aprendiz y sesión. También se espera que otro desarrollador pueda reproducir y mantener la solución a partir de su documentación.

Evidencias de los resultados esperados

Presentar un recorrido real con datos de prueba y registrar: respuesta de acceso; asociación de ficha; enrolamiento persistido; resultado de verificación; evento almacenado; consulta restringida por rol. Para afirmar mejoras cuantitativas, medir antes y después con el mismo escenario. Este documento no inventa porcentajes de precisión, tiempos de respuesta ni pruebas aprobadas.



9.1. Conclusiones

Conclusiones

FaceLit reúne gestión académica y asistencia con reconocimiento facial en una solución de varios componentes. Su backend Java es una aplicación monolítica organizada por módulos; el microservicio Python realiza el procesamiento facial; PostgreSQL relaciona datos académicos y biométricos; Expo presenta las funciones para web y Android.

La calidad de la explicación depende de mantener las mismas definiciones y distinguir requisito, implementación y evidencia. Un endpoint existente no demuestra un caso de uso completo, una pantalla no acredita persistencia y un despliegue descrito no garantiza disponibilidad actual. Esa precisión permite exponer el proyecto con claridad y reconocer sus mejoras pendientes.

Fuentes y criterio de elaboración

Repositorios aportados: FaceLit-DB, FaceLit-Backend, FaceLit y micro-facial; SRS_V3_completo_y_trazable y Despliegue-FaceLit.md. Las imágenes son suministradas por el equipo. El artículo académico aportado se tomó como referencia de presentación, sin trasladar su contenido de domótica al proyecto. Las formulaciones de objetivos y explicación didáctica son redacción de este documento, basada en esas fuentes. Para esta actualización se incorporaron los documentos adjuntos de patrones y frontend y la explicación técnica del backend V5 aportada por el equipo. Los documentos difieren en la descripción de eventos de notificación; se conserva el flujo más específico de envío de correo sin afirmar la ausencia general de eventos.